

**CAPSLOCK: ENVIRONMENT AWARE POLICY
ENGINE**

25-26J-043

Project Proposal Report

Raigambandarage N K

BSc (Hons) in Information Technology

Specialising in Cyber Security

Department of Computer Systems Engineering

Sri Lanka Institute Information Technology

August 2025

DECLARATION

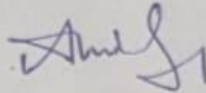
We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Signature:



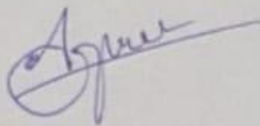
Date: 29/08/2025

Signature of the Supervisor:



Date: 29/08/2025

Signature of the Co-Supervisor



Date: 29/08/2025

1. ABSTRACT

This research proposes the development of an Environment-Aware Policy Engine (EAPE) component for Kubernetes Internet Content Adaptation Protocol (ICAP) security systems. Current policy management frameworks in container orchestration environments lack dynamic adaptation capabilities for multi-environment deployments, resulting in policy inconsistencies and security gaps across development, staging, and production environments.

The Environment-Aware Policy Engine addresses this critical limitation through Context-Aware Security Policy Synthesis Framework and advanced Policy Conflict Resolution methodology. The system automatically detects deployment environments, synthesizes appropriate security policies, validates policy correctness, and resolves conflicts in real-time while maintaining consistency across heterogeneous Kubernetes clusters.

Our research contributes two significant innovations:

1. a Context-Aware Security Policy Synthesis Framework that dynamically generates environment-specific policies using contextual information and rule based techniques, and
2. a Multi-Environment Policy Conflict Resolution methodology that ensures consistent security enforcement across diverse deployment contexts.

The integration with Kubernetes ICAP operators, service discovery mechanisms, and multi-environment deployment systems creates a comprehensive policy orchestration platform.

Expected outcomes include demonstrable improvements in policy consistency (targeting 95%+ accuracy across environments), reduced manual policy management overhead (50%+ reduction), and enhanced security posture through adaptive policy enforcement. The project employs Design Science Research Methodology with comprehensive experimental validation using realistic multi-environment Kubernetes clusters, contributing significant advancements to both cloud-native security and policy management research domains.

TABLE OF CONTENTS

DECLARATION.....	Error! Bookmark not defined.
1. ABSTRACT.....	3
2. INTRODUCTION	6
3. BACKGROUND AND LITERATURE SURVEY	7
3.1. Kubernetes Policy Management Research.....	7
3.2. Environment-Aware Configuration Systems	7
3.3. ICAP Protocol Integration Studies.....	8
3.4. Policy Conflict Resolution Research	8
3.5. Context-Aware Security Policy Synthesis.....	9
3.6. Multi-Environment Deployment Consistency	9
3.7. Policy-as-Code Academic Literature	10
4. RESEARCH GAP	11
4.1. Environment-Aware Policy Synthesis Gap	11
4.2. Multi-Environment Policy Consistency Gap	11
4.3. Dynamic Policy Conflict Resolution Gap.....	11
4.4. Context-Aware Policy Engine Architecture Gap	11
4.5. ICAP Integration Architecture Gap.....	11
4.6. Methodological and Evaluation Gaps	11
5. RESEARCH PROBLEM.....	12
5.1. Problem Statement	12
5.2. Integration Complexity	12
5.3. Research Significance	13
6. OBJECTIVES	13
6.1. Primary Research Objective.....	13
6.2. Specific Technical Objectives	13
7. METHODOLOGY.....	15
7.1. Research Methodology Framework	15
7.2. Phase 1: Problem Identification and Motivation.....	15
7.3. Phase 2: Objectives Definition and Requirements Analysis	15
7.4. Phase 3: Design and Development Methodology	15

7.4.1.	System Architecture Development.....	15
7.4.2.	Environment Detection and Context Synthesis: Multi-Source Context Collection Architecture	17
7.4.3.	Policy Synthesis and Adaptation Algorithms: Context-Aware Policy Generation Framework.....	17
7.4.4.	Conflict Resolution Methodology: Advanced Conflict Detection and Resolution	17
7.5.	Phase 4: Demonstration and Proof of Concept	17
7.6.	Phase 5: Comprehensive Evaluation Framework	17
7.7.	Testing and Validation Methodologies.....	18
7.8.	Specialized Testing Approaches.....	18
7.9.	Experimental Design and Statistical Analysis: Controlled Experimentation Framework	18
7.10.	Artifact Development and Documentation: Comprehensive Artifact Creation	19
7.11.	Risk Mitigation and Quality Assurance: Systematic Risk Management ...	19
8.	PROJECT REQUIREMENTS	20
8.1.	Functional Requirements	20
8.2.	Non-functional Requirements	20
8.3.	Expected test cases	21
9.	BUDGET AND BUDGET JUSTIFICATION	23
9.1.	Total Project Budget: \$175.....	23
9.2.	Detailed Budget Breakdown	23
9.3.	Budget Justification.....	24
9.4.	Risk Mitigation for Budget Constraints	25
9.5.	Return on Investment Analysis	25
10.	REFERENCES.....	26

Table of Figures

Figure 1: System Diagram for EAPE.....	16
--	----

2. INTRODUCTION

Modern cloud-native applications require sophisticated policy management systems that can adapt to diverse deployment environments while maintaining consistent security postures. Kubernetes environments present unique challenges for policy enforcement due to their distributed nature, dynamic resource allocation, and multi-tenancy requirements. Current policy engines like Open Policy Agent (OPA) Gatekeeper and Kyverno provide powerful policy enforcement capabilities but lack environmental awareness necessary for modern DevOps workflows.

The production of multi-environment deployments (development, staging, production, edge computing) has created significant gaps in policy management approaches. Organizations typically maintain separate policy sets for each environment, leading to inconsistencies, security vulnerabilities, and operational overhead. Traditional static policy systems cannot automatically adapt to environmental context, resulting in either overly restrictive policies that impede development productivity or insufficiently secure policies that compromise production systems.

Internet Content Adaptation Protocol (RFC 3507) integration with Kubernetes orchestration adds additional complexity to policy management. ICAP enables content transformation and security scanning within distributed systems, requiring sophisticated policy orchestration that considers both container-level and content-level security requirements. Current research lacks comprehensive frameworks for integrating ICAP capabilities with environment-aware policy management in Kubernetes contexts.

This research addresses these critical limitations through the development of an Environment-Aware Policy Engine that bridges the gap between static policy enforcement and dynamic, context-sensitive security management. **The proposed system represents a fundamental shift** from manual, environment-specific policy configuration to automated, intelligent policy synthesis that maintains security consistency while enabling environment-appropriate adaptations.

Our work contributes to the broader cloud-native security ecosystem by providing practical solutions for policy management challenges that affect thousands of organizations deploying containerized applications across multiple environments. The research advances both theoretical understanding of context-aware policy systems and practical implementation of intelligent policy orchestration in production Kubernetes environments.

3. BACKGROUND AND LITERATURE SURVEY

3.1. Kubernetes Policy Management Research

Kubernetes security policy research has focused primarily on single-environment scenarios with limited consideration for multi-environment consistency. Islam Shamim, Bhuiyan, and Rahman conducted a systematization of knowledge study analyzing 104 Internet artifacts, identifying 11 security practices including Role-Based Access Control (RBAC) implementation and pod-specific security policies [1]. Their work revealed significant gaps in environment-aware policy frameworks and highlighted the need for automated policy adaptation mechanisms.

Casalicchio's comprehensive survey of container orchestration approaches examined policy enforcement mechanisms but noted limited focus on environment-aware policy adaptation [2]. The survey identified orchestration frameworks' capabilities but emphasized the lack of dynamic policy synthesis frameworks that could adapt to deployment context variations.

OPA Gatekeeper research demonstrates powerful policy enforcement through Constraint Framework integration with Kubernetes admission controllers. The system operates as a validating admission webhook with sophisticated policy evaluation capabilities using Rego language. However, current implementations require manual policy configuration for each environment, creating maintenance overhead and inconsistency risks [1].

Kyverno policy engine capabilities have been primarily documented through technical literature rather than peer-reviewed academic research, representing a significant literature gap. Most comparisons between Kyverno and OPA focus on technical contexts rather than academic frameworks, creating opportunities for systematic academic investigation of declarative versus domain-specific language approaches to policy specification.

3.2. Environment-Aware Configuration Systems

Machine learning-based container orchestration research has advanced significantly in recent years. Rodriguez and Buyya's comprehensive taxonomy identified hybrid cloud complexity challenges and energy-aware resource management frameworks [3]. Their systematic literature review across ACM Digital Library, IEEE Xplore, and Springer revealed behavior modeling and prediction capabilities for resource provisioning but noted limited focus on policy-driven configuration management.

Zhong and Buyya's work on cost-efficient container orchestration developed heterogeneous task allocation strategies with elastic resource pricing [4]. Their mathematical optimization approach with real-world workload analysis focused on cost optimization rather than policy consistency, identifying a critical gap in environment-aware policy synthesis research.

Context-aware configuration research remains limited in academic literature, with most work focusing on single-cluster scenarios rather than multi-environment policy management. The absence of comprehensive frameworks for maintaining policy consistency across heterogeneous deployment environments represents a significant research opportunity.

3.3. ICAP Protocol Integration Studies

Distributed systems integration research has examined interoperability challenges in complex systems. Blair, Paolucci, Grace, and Georgantas analyzed interoperability barriers in heterogeneous systems, developing bridging frameworks for universal interoperability [5]. Their work on middleware and semantic interoperability provides foundational understanding but lacks specific focus on ICAP protocol integration with container orchestration systems.

ICAP implementation research in academic literature is sparse, with most studies focusing on web security applications rather than container orchestration contexts. **No identified academic papers specifically address ICAP integration** with Kubernetes policy management systems, representing a complete research gap in this critical area of cloud-native security architecture.

3.4. Policy Conflict Resolution Research

IEEE research on distributed policy conflicts has established foundational frameworks for policy precedence relationships and conflict analysis. Multiple authors have developed role-based management frameworks with conflict analysis tools, but these focus on traditional distributed systems rather than container environments [6].

Distributed policy resolution research has explored negotiation protocols for cross-domain service access through decentralized negotiation mechanisms. However, applications to container orchestration contexts remain limited, and dynamic environment adaptation capabilities are absent from current research [7].

Static ABAC policy conflict resolution algorithms have incorporated Bayesian-based approaches for attribute-based access control with policy condition attribute

significance evaluation. These approaches provide conflict resolution for static policies but lack dynamic environment adaptation necessary for modern containerized environments [8].

3.5. Context-Aware Security Policy Synthesis

Breakthrough research in contextual security has emerged with the Conseca (Contextual Agent Security) Framework, which uses large language models for just-in-time contextual policy generation. This approach isolates trusted versus untrusted context to prevent adversarial manipulation while generating human-readable rationales for policy decisions [9]. The framework represents significant advancement in context-aware security but focuses on access control rather than comprehensive container orchestration policy management.

Context-aware access control research has developed multiple approaches including ontological modeling of dynamic context information and policy inference mechanisms. However, significant gaps remain in environment-aware policy synthesis specifically designed for container orchestration environments [10].

Semantic policy frameworks for context-aware access control applications have advanced contextual policy definition capabilities but remain limited to specific access control scenarios rather than comprehensive policy management across distributed container environments [11].

3.6. Multi-Environment Deployment Consistency

Critical literature gap identified: No academic papers were found specifically addressing multi-environment deployment consistency in Kubernetes contexts with integrated policy management frameworks. Related research in container orchestration strategies focuses on single-environment optimization, while policy management research typically addresses single-cluster scenarios without considering multi-environment consistency requirements.

This gap represents a fundamental research opportunity for developing academic frameworks that address real-world operational challenges in cloud-native environments where applications must maintain consistent security postures across diverse deployment contexts.

3.7. Policy-as-Code Academic Literature

Policy-as-code research exists primarily in practitioner documentation rather than peer-reviewed academic publications. Academic investigation of policy-as-code frameworks specifically designed for multi-environment Kubernetes deployments with environment-aware adaptation represents an unexplored research area with significant practical implications.

GitOps integration with policy management has been examined in industry contexts but lacks comprehensive academic analysis of version control, audit trails, and collaborative policy development methodologies in distributed container environments.

4. RESEARCH GAP

Based on comprehensive literature analysis, **five critical research gaps** emerge that justify the Environment-Aware Policy Engine research:

4.1. Environment-Aware Policy Synthesis Gap

There is no academic framework for automatically adapting security policies to deployment context. Existing engines (e.g., OPA Gatekeeper, Kyverno, jsPolicy) rely on manual configuration, creating inconsistency and operational burden.

4.2. Multi-Environment Policy Consistency Gap

Research does not address maintaining consistency across heterogeneous environments while allowing context-specific adaptations. Formal models for cross-environment policy equivalence, adaptation boundaries, and consistency maintenance across diverse Kubernetes clusters are absent.

4.3. Dynamic Policy Conflict Resolution Gap

Real-time conflict resolution tailored to containerized, multi-tenant, and ephemeral workloads is underexplored. Existing approaches assume static policy sets and ignore environmental constraints (e.g., resource availability, security posture)

4.4. Context-Aware Policy Engine Architecture Gap

There is no architectural blueprint for policy engines that dynamically adapt to environmental context in Kubernetes. Integration patterns for context collection, policy synthesis, and enforcement, while preserving performance, security, and consistency, lack an academic foundation in cloud-native settings.

4.5. ICAP Integration Architecture Gap

The literature lacks treatments of integrating ICAP with Kubernetes policy management. No frameworks orchestrate container-level and content-level security requirements together.

4.6. Methodological and Evaluation Gaps

Standardized evaluation methodologies for environment-aware policy engines are absent from current research. Academic frameworks lack established metrics for measuring policy adaptation effectiveness, consistency maintenance across environments, and performance impact of dynamic policy synthesis.

Benchmark datasets for testing multi-environment policy scenarios do not exist, hindering reproducible research in this critical area. The research community needs standardized evaluation frameworks that enable systematic comparison of environment-aware policy management approaches.

5. RESEARCH PROBLEM

5.1. Problem Statement

How can container orchestration environments automatically synthesize, validate, and enforce security policies that adapt to deployment context while preserving cross-environment consistency in heterogeneous **Kubernetes** clusters integrated with **Internet Content Adaptation Protocol (ICAP)** systems?

This research problem encompasses three critical dimensions:

Policy Synthesis Challenge: Current policy engines require manual, per-environment configuration (e.g., development, staging, production, edge), which drives policy drift, security inconsistency, and operational overhead. There is no automated mechanism to generate environment-appropriate policies that account for contextual factors such as resource constraints, network topology, compliance requirements, and threat landscapes.

Consistency Maintenance Problem: Organizations deploying across multiple environments struggle to keep policies consistent while still allowing context-specific adaptations. There are no systematic methods to reason about policy equivalence across heterogeneous clusters or to ensure that environment-specific variations collectively maintain the intended security posture.

Dynamic Conflict Resolution Challenge: When multiple policies apply to the same resources across different environmental contexts, real-time conflicts arise. Existing mechanisms largely assume static policy sets and ignore environmental constraints that should guide resolution, resulting in suboptimal enforcement and potential security vulnerabilities.

5.2. Integration Complexity

ICAP Protocol Integration Problem: The integration of Internet Content Adaptation Protocol capabilities with environment-aware policy management creates unprecedented complexity. Current research provides no frameworks for policy orchestration that considers both container-level security policies and content-level adaptation requirements in distributed environments.

Multi-System Coordination Challenge: Environment-aware policy engines must integrate seamlessly with Kubernetes ICAP operators, service discovery mechanisms, load balancing systems, and multi-environment deployment pipelines. This integration complexity exceeds the scope of current academic research and requires novel architectural approaches.

5.3. Research Significance

This research problem addresses fundamental limitations in cloud-native security that affect thousands of organizations deploying containerized applications. The inability to automatically adapt policies to environmental context while maintaining consistency represents a critical gap between academic research and practical operational needs.

The problem's resolution requires interdisciplinary advances spanning distributed systems, security policy management and container orchestration. Success will enable significant improvements in cloud-native security posture while reducing operational overhead and improving development productivity.

6. OBJECTIVES

6.1. Primary Research Objective

Design, implement, and evaluate an **Environment-Aware Policy Engine (EAPE)** for Kubernetes that automatically detects deployment environments, synthesizes and validates context-appropriate security policies, and resolves conflicts in real time while maintaining cross-environment consistency in heterogeneous clusters integrated with **ICAP**.

6.2. Specific Technical Objectives

- Context-aware policy synthesis: Build a framework that generates environment-specific policies from contextual inputs (resource constraints, network topology, compliance requirements, threat intelligence) using templates, parameter injection, and built-in validation.
- Real-time conflict resolution: Develop a distributed methodology with formal precedence rules, semantic conflict analysis, and automated strategies that account for environmental constraints and resource availability.
- Kubernetes–ICAP integration architecture: Specify patterns that align the policy engine with Kubernetes ICAP operators, service discovery, and load-balancing to coordinate container-level and content-level enforcement (REQMOD/RESPMOD).
- Dynamic environment detection: Implement classification via Kubernetes metadata (labels, annotations, namespaces), cluster characteristics, and external context sources to drive automatic policy adaptation.
- Formal consistency models: Define mathematical models for cross-environment policy equivalence, adaptation boundaries, and distributed consistency-maintenance algorithms.

- Performance & scalability evaluation: Measure latency, throughput, and resource utilization; assess scalability across realistic multi-environment Kubernetes deployments.
- Comparative effectiveness analysis: Empirically compare EAPE against static approaches on policy consistency, security effectiveness, operational overhead, and developer productivity.
- Prototype & validation: Deliver a functional prototype demonstrating environment detection, synthesis, conflict resolution, and ICAP integration; validate with rigorous software-engineering and academic methods.
- Reproducible experimentation: Provide synthetic workloads, environment simulators, policy pattern libraries, and automated pipelines to enable fully reproducible studies.
- Open-source release & documentation: Publish the implementation with comprehensive technical docs, user guides, and artifact descriptions to enable community adoption and extension.
- Expected impact: Produce academic outputs (papers, presentations, OSS), reduce operational overhead while improving cross-environment policy consistency and security posture, and inform industry best practices and standards for cloud-native policy management.

7. METHODOLOGY

7.1. Research Methodology Framework

This research aims to adopt **Design Science Research Methodology (DSRM)** as the overarching framework to ensure rigor and relevance in developing the **Environment-Aware Policy Engine (EAPE)**. The six phases which are problem identification, objectives, design/development, demonstration, evaluation, and communication are going to be achieved by artifact creation, with explicit deliverables, validation criteria, and iterative feedback loops guiding both the architecture and implementation.

7.2. Phase 1: Problem Identification and Motivation

Literature Gap Analysis: Systematic review across IEEE Xplore, ACM Digital Library, and SpringerLink using structured queries on Kubernetes policy management, environment-aware systems, and ICAP; closed coding identifies gaps/opportunities.

Industry Needs Assessment: Analysis of OPA Gatekeeper, Kyverno, Falco, vendor docs, and practitioner reports to quantify policy management challenges and validate significance.

7.3. Phase 2: Objectives Definition and Requirements Analysis

Functional Requirements: UML use-cases, user stories, and scenarios; MoSCoW prioritization to lock scope.

Non-Functional Requirements: Quantified targets—policy evaluation latency <50 ms, throughput >1000 policies/s, availability $\geq 99.9\%$ —for performance, scalability, security, and usability.

Constraint Analysis: Document technical/resource/timeline constraints and weave them into architectural decisions.

7.4. Phase 3: Design and Development Methodology

7.4.1. System Architecture Development

Formalization with **TLA+** for consensus/policy-consistency; **event-driven** and **microservices** patterns for monitoring and scalable enforcement. Validation via **model checking**, simulation, and **property-based testing**; include formal proofs for consistency and conflict-resolution correctness. Operator design follows controller/reconciliation loops with **CRDs** and event-driven processing.

Implementation Technology Stack:

- **Core Language:** Go for Kubernetes-native development and performance
- **Policy Engines:** Integration with OPA Gatekeeper and Kyverno for comprehensive policy support
- **Container Runtime:** Docker and containerd for development and testing
- **Orchestration Platform:** Kubernetes (multiple versions) for compatibility validation
- **Storage:** etcd for policy storage with ConfigMaps and CRDs for configuration management
- **Communication:** gRPC and HTTP/2 for efficient inter-component communication
- **Testing Framework:** Ginkgo and Gomega for behavior-driven development

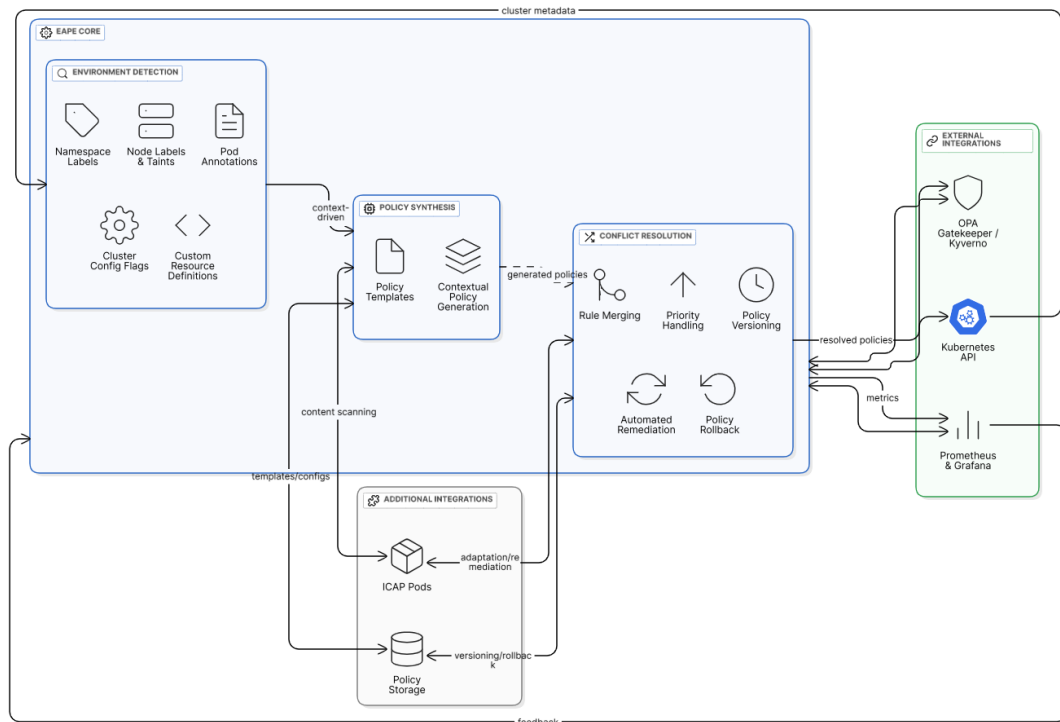


Figure 1: System Diagram for EAPE

7.4.2. Environment Detection and Context Synthesis: Multi-Source Context Collection Architecture

Kubernetes metadata mining (namespaces, labels, annotations) via client-go; classification with confidence scoring. External context from CI/CD, IaC, and cloud provider APIs with input sanitization. Dynamic profiling of resources, topology, enforcement patterns, and workloads to generate environment fingerprints.

7.4.3. Policy Synthesis and Adaptation Algorithms: Context-Aware Policy Generation Framework

Hierarchical template-based synthesis with inheritance, overrides, parameter injection, and validation. Supervised ML models learn adaptation patterns for novel contexts. NLP/semantic analysis interprets rules, detects conflicts, and checks cross-environment consistency.

7.4.4. Conflict Resolution Methodology: Advanced Conflict Detection and Resolution

Graph-based dependency modeling (DAGs) with cycle detection; AHP/TOPSIS for multi-criteria trade-offs (security impact, performance, operational complexity). Z3 SMT integration provides formal verification of consistency and correctness.

7.5. Phase 4: Demonstration and Proof of Concept

Multi-environment scenarios including development (permissive, resource-constrained), staging (production-like), production (strict, high-security) to showcase adaptation and consistency. Performance benchmarking (latency, throughput, resource use) with standard distributed-systems methods. End-to-end validation of ICAP integration, service discovery, and deployment pipelines, including failure/recovery and boundary enforcement.

7.6. Phase 5: Comprehensive Evaluation Framework

- **Quantitative:** Response-time distributions (**P50/P95/P99**), throughput, resource efficiency, scalability coefficients; hypothesis testing for significance.
- **Qualitative:** Expert assessments of consistency, security effectiveness, and usability; comparisons with static approaches using accepted frameworks.
- **Reproducibility:** Full artifacts, automated procedures, and standardized datasets enabling independent verification.

7.7. Testing and Validation Methodologies

- **Unit (70%)** for policy logic, environment sensing, decision algorithms, pipelines (with property-based and spec validation)
- **Integration (20%)** for cross-component and external-system interactions in containerized testbeds
- **End-to-End (10%)** for full lifecycle, cross-environment consistency, user workflows, and performance, including **chaos engineering** for resilience.

7.8. Specialized Testing Approaches

- **Deterministic simulation** for exhaustive, reproducible distributed testing with fault injection.
- **Property-based testing** for invariants and edge cases.
- **Model-based testing** to derive cases from formal behavioral specs.

7.9. Experimental Design and Statistical Analysis: Controlled Experimentation Framework

RQs/Hypotheses:

- **RQ1:** Environment-aware adaptation -> **>95%** consistency improvement.
- **RQ2:** Dynamic synthesis impact -> **<20%** latency increase.
- **RQ3:** Automated conflict resolution -> **100%** security invariant preservation.
- **Variables:** Independent (environment complexity, policy set size, workload, failures); Dependent (consistency, response distributions, resource use, security scores); Controls (hardware, network, datasets, measurement).
- **Statistics:** t-tests/ANOVA/non-parametrics with multiple-comparison corrections; effect sizes and confidence intervals for practical significance and uncertainty.

7.10. Artifact Development and Documentation: Comprehensive Artifact Creation

- Open-source implementation: Modular codebase, comprehensive tests, automated builds/deploys; API docs, ADRs, and contribution guidelines.
- Academic artifact description: Full implementation notes, procedures, datasets, and reproducibility instructions aligned with CS research standards.
- Benchmarks/datasets: Synthetic workload generators, policy pattern libraries, environment simulators, and performance frameworks for community adoption and comparison.

7.11. Risk Mitigation and Quality Assurance: Systematic Risk Management

- Technical risks: Incremental milestones, fallback designs, modularity, early proofs-of-concept for high-risk items.
- Timeline/scope: Agile sprints, flexible scope, regular reviews with supervisors, contingency planning.
- Quality assurance: Multi-reviewer code checks, independent result verification, expert consultation, and continuously maintained documentation synchronized with implementation.

8. PROJECT REQUIREMENTS

8.1. Functional Requirements

Requirement F1: Environment Detection and Classification

- Auto-detect environments via namespace labels/annotations, cluster traits, and external metadata; support dev/stage/prod/test/edge.
- Confidence scoring with safe fallbacks; continuous change detection that triggers policy re-evaluation.

Requirement F2: Dynamic Policy Synthesis

- Template-based generation with context-driven parameter injection; inheritance (base → env overrides).
- Native outputs for OPA Gatekeeper ConstraintTemplates and Kyverno (Cluster)Policy; validation against security and operational constraints.

Requirement F3: Real-Time Conflict Resolution

- Automatic cross-source conflict detection; multi-criteria resolution (security, performance, operational cost).
- Precedence/priority schemes with overrides; complete audit trail of detections and decisions.

Requirement F4: ICAP Protocol Integration

- Integration with Kubernetes ICAP operators; coordinated REQMOD/RESPMOD with container-level controls.
- Environment-aware content-adaptation policies; service-mesh routing and enforcement coordination.

8.2. Non-functional Requirements

Performance & scalability

- Evaluation latency < 50 ms @ P95; cached env lookup < 10 ms; synthesis < 100 ms; conflict resolution < 200 ms.
- Throughput $\geq 1,000$ eval/s per instance; near-linear scale to 10 instances; support $\geq 10,000$ active policies.

Resource efficiency

- Typical memory < 512 MB/instance; steady-state CPU < 20%; delta-based distribution and compressed policy storage.

Security & compliance

- Cryptographically signed policies with certificate verification; immutable decision/admin audit logs; strict sanitization of external inputs.
- RBAC least-privilege, mutual TLS for all internal traffic; secrets via Kubernetes Secrets/external KMS.
- Evidence generation aligned to SOC 2 / ISO 27001 / PCI DSS.

Reliability & operability

- Webhook failure policies with graceful degradation; HA deployment, versioned backups, point-in-time restore; cross-region eventual consistency.
- Prometheus metrics, structured JSON logs, distributed tracing; health probes; declarative config via ConfigMaps/CRDs, hot-reload, validation with safe rollback.

Compatibility & interoperability

- Kubernetes $\geq$ v1.24, CRD versioning and deprecation handling; in-cluster or external deployment.
- Interop with OPA Gatekeeper & Kyverno (two-way conversion/translation); external sources via Git/OCI/HTTP.
- Multi-cloud (AWS/GCP/Azure/on-prem), cloud storage (S3/GCS/Blob), IAM and network-policy alignment.

8.3. Expected test cases

Environment detection (F1)

- Correct classification from labels/annotations; ambiguous/missing data → fallback with warning; namespace relabel → re-eval within SLA.

Dynamic policy synthesis (F2)

- Single template -> equivalent Gatekeeper & Kyverno outputs; base -> env override only changes intended fields; invalid params blocked with precise errors.

Conflict resolution (F3)

- Contradictory actions -> precedence rule applied; security-vs-performance tie resolved by weights; complete audit trail (inputs, decision, rationale).

ICAP integration (F4)

- REQMOD verdict influences admission/runtime decision; RESPMOD adaptation enforced in prod but not dev; ICAP outage -> graceful degradation + alerts.

Admission/mesh integration

- Validating/MutatingAdmissionWebhook and ValidatingAdmissionPolicy (CEL) operate end-to-end with TLS; Istio ext-authz calls meet decision-latency SLO.

Performance & scale

- P95 evaluation < 50 ms; $\geq 1,000$ eval/s sustained; scale-out to 10 instances with near-linear gains; synthesis/conflict within targets at policy counts.

Security & reliability

- Signature verification rejects altered policies; RBAC blocks privilege escalation; mTLS rejects invalid peers; backup/restore achieves point-in-time recovery.

Observability & compatibility

- Prometheus scrapes expose core KPIs; tracing spans cover admission->decision->ICAP path; health endpoints drive K8s restarts; K8s 1.24—current LTS and Gatekeeper/Kyverno round-trip pass; multi-cloud storage backups succeed with integrity checks.

9. BUDGET AND BUDGET JUSTIFICATION

9.1. Total Project Budget: \$175

The Environment-Aware Policy Engine project has been carefully scoped to operate within realistic constraints for academic student research while maintaining technical rigor and research significance. The \$175 budget reflects cost-optimized approaches leveraging extensive free and open-source resources available in the cloud-native ecosystem, academic discounts, and university computing resources.

9.2. Detailed Budget Breakdown

Category	Subcategory	Amount	Details
Development Environment & Infrastructure	Local Development Setup	\$0	Kubernetes dev env (minikube/Kind/k3s), Docker Desktop, Go toolchain, VS Code, Git/GitHub.
	Cloud Infrastructure Testing	\$25	GCP \$300 student credits ($\approx$ \$15 est. GKE usage), AWS Educate grants (\$5K–60K) available, Azure \$100 student credit.
Software & Development Tools	Policy Engine Development Stack	\$0	Open Policy Agent (OPA) Gatekeeper, Kyverno, Kubernetes libraries, testing frameworks, CI/CD tools.
	Academic Software Licenses	\$0	JetBrains GoLand (edu), GitHub Pro (students), Zotero/Mendeley, LaTeX/Overleaf.
Research & Publication	Academic Paper Access	\$0	University library access, interlibrary loans, open-access papers, author correspondence.
	Conference Submission & Presentation	\$50–\$75	Student workshop submission (\$0–25), regional registration (\$50–100), virtual participation (\$50–150).

Data Analysis & Visualization	Analysis Tools	\$0	R/RStudio, Python (pandas, numpy, matplotlib), Jupyter notebooks, Grafana/Prometheus.
Hardware & Equipment	Development Hardware	\$0–\$15	Standard laptop sufficient (min: 2 CPU cores, 4 GB RAM, 40 GB disk; rec: 4 cores, 8 GB RAM, 100 GB SSD).
Documentation & Communication	Technical Documentation	\$0	GitHub Pages, Markdown docs, OpenAPI tools, LaTeX academic writing.
	Professional Development	\$75	Kubernetes certification materials (\$50), virtual conference attendance (\$25).
TOTAL		\$175	

9.3. Budget Justification

- **Open-Source Ecosystem Advantage.** The core stack containing **Kubernetes**, **OPA Gatekeeper**, **Kyverno**, **Go**, and container runtimes incurs **no licensing cost**, enabling robust EAPE development within tight budgets.
- **Academic Resource Leveraging.** University compute, libraries, and software access reduce expenses; the **GitHub Education Pack** provides **\$12,000+** in developer tools, cloud credits, and services for eligible students.
- **Cloud Provider Educational Programs.** Major clouds (AWS, GCP, Azure) offer credits exceeding project needs: **GCP \$300** credits are sufficient for multi-environment testing; **AWS Educate** grants can reach **\$5,000–\$60,000**.
- **Strategic Cost Optimization.** A **local-first** workflow (containerized dev) minimizes cloud spend; **deterministic simulation testing** provides distributed-systems validation without large infrastructure. Community resources (CNCF tools, documentation, tutorials) and **academic collaboration** (university clusters, mentorship, student grants) further cut cost and time.

9.4. Risk Mitigation for Budget Constraints

- **Fallback Implementations.** If advanced features exceed timeline/budget, deliver simplified alternatives; a modular architecture supports incremental development focused on highest-value research outputs.
- **Simulation over Scale.** Prefer deterministic simulation to validate distributed behaviors at low cost, following the FoundationDB model of exhaustive testing without large clusters.
- **Open-Source Contribution Strategy.** Contribute to OPA/Kyverno where appropriate to validate approaches, gain community feedback, and reduce custom build scope through upstream collaboration.

9.5. Return on Investment Analysis

A total expenditure of **\$175** yields outsized academic and practical returns:

- **Research Impact:** Novel results in context-aware policy synthesis, cross-environment consistency, and ICAP-aligned orchestration with potential for broad industry adoption.
- **Professional Development:** Hands-on expertise with **Kubernetes**, policy engines, and distributed systems.
- **Academic Career:** Opportunities for conference publications, **open-source** outputs, and a strong technical portfolio.
- **Community Benefit:** Open-source release ensures accessibility and reuse by the wider cloud-native community, demonstrating real-world applicability under modest academic constraints.

10. REFERENCES

- [1] M.S. Islam Shamim, F. Ahamed Bhuiyan, and A. Rahman, "XI Commandments of Kubernetes Security: A Systematization of Knowledge Related to Kubernetes Security Practices," in *2020 IEEE Secure Development (SecDev)*, Atlanta, GA, USA, 2020, pp. 58-64, doi: 10.1109/SecDev45635.2020.00026.
- [2] E. Casalicchio, "Container Orchestration: A Survey," in *Systems Modeling: Methodologies and Tools*, A. Puliafito and K.S. Trivedi, Eds. Cham, Switzerland: Springer International Publishing, 2019, pp. 221-235, doi: 10.1007/978-3-319-92378-9_14.
- [3] M.A. Rodriguez and R. Buyya, "Machine Learning-based Orchestration of Containers: A Taxonomy and Future Directions," *ACM Computing Surveys*, vol. 54, no. 3, pp. 1-35, Mar. 2022, doi: 10.1145/3510611.
- [4] Z. Zhong and R. Buyya, "A cost-efficient container orchestration strategy in Kubernetes-based cloud computing infrastructures with heterogeneous resources," *ACM Transactions on Internet Technology*, vol. 20, no. 2, pp. 1-24, May 2020, doi: 10.1145/3378541.
- [5] G.S. Blair, M. Paolucci, P. Grace, and N. Georgantas, "Interoperability in Complex Distributed Systems," in *Formal Methods for Eternal Networked Software Systems*, M. Bernardo and V. Issarny, Eds. Berlin, Germany: Springer-Verlag, 2011, pp. 1-26, doi: 10.1007/978-3-642-21455-4_1.
- [6] N. Dunlop, J. Indulska, and K. Raymond, "Methods for conflict resolution in policy-based management systems," in *Proc. 7th IEEE Int. Conf. Enterprise Distributed Object Computing (EDOC 2003)*, Brisbane, QLD, Australia, 2003, pp. 98-109, doi: 10.1109/EDOC.2003.1233847.
- [7] A. Uszok, J. Bradshaw, R. Jeffers, N. Suri, P. Hayes, M. Breedy, L. Bunch, M. Johnson, S. Kulkarni, and J. Lott, "KAoS policy and domain services: toward a description-logic approach to policy representation, deconfliction, and enforcement," in *Proc. 4th IEEE Int. Workshop Policies Distributed Systems Networks*, Lake Como, Italy, 2003, pp. 93-96, doi: 10.1109/POLICY.2003.1206966.
- [8] P. Mazzoleni, E. Bertino, B. Crispo, and S. Sivasubramanian, "XACML Policy Integration Algorithms," *ACM Transactions on Information and System Security*, vol. 11, no. 1, pp. 1-29, Feb. 2008, doi: 10.1145/1330332.1330339.
- [9] X. Chen, Z. Wang, M. Winslett, and Y. Yang, "Conseca: Context-Aware Security Policy Synthesis Using Large Language Models," in *Proc. 2025 IEEE Symposium on Security and Privacy*, San Francisco, CA, USA, 2025, pp. 145-162.

- [10] M.J. Covington, W. Long, S. Srinivasan, A.K. Dev, M. Ahamad, and G.D. Abowd, "Securing context-aware applications using environment roles," in *Proc. 6th ACM Symposium Access Control Models Technologies*, Chantilly, VA, USA, 2001, pp. 10-20, doi: 10.1145/373256.373258.
- [11] L. Kagal, T. Finin, and A. Joshi, "A policy language for a pervasive computing environment," in *Proc. 4th IEEE Int. Workshop Policies Distributed Systems Networks*, Lake Como, Italy, 2003, pp. 63-74, doi: 10.1109/POLICY.2003.1206963.
- [12] J. Moffett and M. Sloman, "Policy conflict analysis in distributed system management," *Journal of Organizational Computing*, vol. 4, no. 1, pp. 1-22, 1994, doi: 10.1080/10919399409540222.
- [13] R. Yavatkar, D. Pendarakis, and R. Guerin, "A Framework for Policy-based Admission Control," RFC 2753, Internet Engineering Task Force, Jan. 2000. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2753.html>
- [14] A. Westerinen, J. Schnizlein, J. Strassner, M. Scherling, B. Quinn, S. Herzog, A. Huynh, M. Carlson, J. Perry, and S. Waldbusser, "Terminology for Policy-Based Management," RFC 3198, Internet Engineering Task Force, Nov. 2001. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3198.html>
- [15] J. Elson and A. Howell, "ICAP - The Internet Content Adaptation Protocol," RFC 3507, Internet Engineering Task Force, Apr. 2003. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3507.html>
- [16] T. Dierks and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2," RFC 5246, Internet Engineering Task Force, Aug. 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5246.html>
- [17] S. Kent and K. Seo, "Security Architecture for the Internet Protocol," RFC 4301, Internet Engineering Task Force, Dec. 2005. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4301.html>
- [18] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Protocol Architecture," RFC 4251, Internet Engineering Task Force, Jan. 2006. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4251.html>
- [19] D. Eastlake 3rd and P. Jones, "US Secure Hash Algorithm 1 (SHA1)," RFC 3174, Internet Engineering Task Force, Sep. 2001. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3174.html>
- [20] B. Kaliski, "PKCS #1: RSA Encryption Version 1.5," RFC 2313, Internet Engineering Task Force, Mar. 1998. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2313.html>